KNOWLEDGE DISTILLATION AND DEPLOYMENT OF A LIGHTWEIGHT COMMAND-PARSING MODEL

Cloud Computing Assignment
Technical Report

Submitted by
Abhimanyu Chauhan
Roll Number: 230101054
B.Tech, Computer Science and Engineering
IIIT Manipur

GitHub Repository
https://github.com/Abhimanyu-Space/cloud-command-parser-distillation

Abstract

This project explores a local command-parsing service built through knowledge distillation. A larger teacher model, Phi-3.5, was used to generate structured labels, and a smaller Pythia-70M model was fine-tuned to produce JSON command frames. The student model was exposed through a FastAPI service and tested with Docker, a local Kubernetes environment, network-rule simulation and Locust load testing.

The work demonstrates an end-to-end prototype, but it does not satisfy every infrastructure and dataset constraint in the assignment. This report describes the implementation and its measured results, while identifying requirements that remain incomplete. It does not present the prototype as a fully compliant production deployment.

1. Project Overview and Architecture

The objective of this project was to convert natural-language commands into a fixed four-field JSON structure using a compact student model. The workflow began with a larger teacher model generating structured labels. These labels were checked and used to fine-tune a smaller model. The trained model was then made available through a FastAPI service, packaged with Docker and tested in a local Kubernetes environment.

Phi-3.5 was used as the teacher model and Pythia-70M as the student model. The project also included scripts for evaluation and packet-rule simulation, deployment configuration files, and a Locust test for sending requests to the API.

2. Dataset Preparation and Teacher Labelling

The teacher-labelling workflow used a pool of 3,000 command examples. The job was divided into two shards of 1,500 records and completed using two Kubernetes jobs. The resulting audit reported 2,972 structurally valid, unique labelled records. A total of 62 unique malformed responses were recorded for review.

The audit categories overlapped: 32 responses had missing fields, 22 had invalid intents and 9 had extra fields. These category counts should not be added together because a single response could contain more than one issue. The validation set contained 100 examples and was kept separate from the teacher-labelled training pool.

The current repository stores project data under the data/ directory. The assignment's exact cleaned-dataset deliverables, datasets/v1/train.jsonl and train.sha256, are not present in the implementation described here. This is one of the data-preparation requirements that remains to be completed.

One important lesson from this stage was that teacher-generated labels cannot be treated as correct automatically. They need schema checks, intent validation, duplicate handling and a review process before they are used for training.

3. Student Model Training

The student model used in this project was EleutherAI Pythia-70M. Training used sequence-level targets, meaning that the model learned to generate the structured JSON response. Prompt tokens were masked with -100 so that they did not contribute to the training loss.

The recorded training run used CPU execution, three epochs, a learning rate of 1e-5, batch size 4, gradient accumulation of 4 and seed 42. It used 2,972 training examples and completed 186 optimizer updates. The recorded target-token count was 63,552 per epoch.

The training loss decreased from 0.4563 in the first epoch to 0.0897 in the second epoch and 0.0559 in the third epoch. This shows that the model fit the training examples over the run, but a lower training loss alone does not establish that the model generalizes well to unseen commands.

There is an important difference between this experiment and the assignment specification. The assignment asks for training on the first 1,500 cleaned training records, whereas this run used 2,972 records. Therefore, the run should be treated as an experiment and not as evidence that the 1,500-record requirement was met. A separate compliant run and the requested model-version comparison remain to be completed.

4. Model Evaluation

The trained student model was evaluated on 100 validation examples. The evaluation reported 100% valid JSON output, 77% intent accuracy, 85% date accuracy, 84% time accuracy, 86% place accuracy and 50% full-frame accuracy.

These results show that the model was able to produce syntactically valid JSON for all examples in this validation sample. However, only half of the complete frames matched all required fields. This is an important distinction: valid formatting does not necessarily mean that the intent and extracted slot values are all correct.

The recorded evaluation covers the trained student only. Teacher and untrained-student baselines were not available in the evaluation evidence. A more complete comparison should include those baselines and should be run on the same validation examples.

5. API Service and Containerization

A FastAPI application was implemented with a POST /parse route for command parsing. The response includes the parsed frame and model/service metadata headers. The current service identifies the model version as full2972_v2 and reports a local serving identifier. A /health route is also implemented.

The current API does not yet match the full interface required by the assignment. The /healthz and /readyz endpoints are not implemented, readiness is not tied to model-loading state as specified, and response validation does not enforce the complete strict four-key contract with the required invalid_model_output error response. The model is loaded from a local path rather than downloaded from object storage at startup.

Docker was used to package the API and model environment. The current Dockerfile copies the model directory into the image. The recorded image occupied approximately 2.29 GB on disk, with a compressed size of approximately 612 MB. Bundling model weights into the image differs from the intended external model-artifact workflow and affects image size and deployment flexibility.

6. Kubernetes Deployment

A local k3d Kubernetes cluster named cmdparser was used. The active context was k3d-cmdparser and the application namespace was pipeline. A deployment and ClusterIP service were created. The service exposed port 80 and forwarded traffic to container port 8080. The current deployment used one replica and a health probe targeting /health.

This is a basic local deployment. The assignment's full requirements for ingress routing, two replicas, horizontal pod autoscaling, rolling update and rollback evidence, pod-deletion recovery and repeated cold-start measurements have not been demonstrated in the recorded implementation. CPU and memory usage per pod were also not captured using kubectl top.

7. Network Access Control

A sample stateless network access control list and packet simulator were created. The sample inbound rules allow a specific TCP/443 source and then deny remaining traffic. Outbound examples include TCP/8000 and a client reply port, followed by a deny rule. Six sample packets were evaluated successfully by the simulator.

This demonstrates rule ordering and stateless packet filtering, but it is not the complete assignment policy. The required subnet plan for 10.20.0.0/16, including app, public, data and management subnets, is not present in the current evidence. The full inbound and outbound policy, including the specified blocked source range, SSH restriction, ephemeral reply ports and application flow, still needs to be implemented and tested.

8. CI/CD and Automation

The repository contains application, training, evaluation, deployment and network-related files. However, a complete GitHub Actions workflow with linting, unit tests, a model evaluation gate, secret scanning and a demonstrated failing run has not yet been implemented. A Makefile-based fresh-clone workflow and an AI-use disclosure file also remain outstanding.

9. Load Testing and Performance

Locust was used to send requests to the parsing API. The recorded run used four users for approximately 120 seconds. The load-test summary reported 129 successful POST requests and zero failed requests. The average response time was approximately 3,605 ms, the median was 3,500 ms, the 95th percentile was 4,400 ms, the 99th percentile was 5,600 ms, the minimum was 2,547 ms and the maximum was 6,368 ms. Throughput was approximately 1.09 requests per second.

There is a discrepancy between the summary count and one CSV statistics row, which records 128 requests while the summary reports 129. The summary result is reported here, but the inconsistency should be checked against the original test output. Per-pod CPU and memory measurements and a cost calculation using the supplied rate card were not completed.

10. Limitations and Future Work

The most important remaining work is to prepare the cleaned dataset in the exact required format and generate its SHA-256 manifest. The student model should then be trained on the specified first 1,500 records, with teacher and untrained-student baselines added to the evaluation.

The model artifact should be moved to the required object storage and downloaded by the API at startup, instead of being copied into the Docker image. The API also needs strict response validation, the required health and readiness endpoints, and the specified error response for invalid model output.

The Kubernetes deployment needs the remaining scaling, ingress, rollout, rollback and recovery tests. The network configuration needs the complete subnet plan and all required stateless rules. Finally, the project needs automated CI/CD checks, unit tests, secret scanning and the cost and service-level objective analysis requested in the assignment.

11. Conclusion

This project demonstrates a working path from teacher-generated labels to a fine-tuned lightweight model and an API that can be containerized and exercised with load testing. The evaluation shows that valid JSON does not guarantee complete semantic correctness. Several cloud-infrastructure and reproducibility requirements remain incomplete. The current result should therefore be treated as a prototype with documented evidence and clear next steps, rather than a fully compliant production deployment.

References

The following documentation and model resources are relevant to the tools used in this project. The numerical results in this report are taken from the project's recorded training, evaluation and load-test outputs.

EleutherAI, Pythia model family: https://github.com/EleutherAI/pythia

Microsoft, Phi-3 model family: https://huggingface.co/microsoft

FastAPI documentation: https://fastapi.tiangolo.com/

Docker documentation: https://docs.docker.com/

Kubernetes documentation: https://kubernetes.io/docs/

Locust documentation: https://docs.locust.io/

k3d documentation: https://k3d.io/

Project repository: https://github.com/Abhimanyu-Space/cloud-command-parser-distillation

This report reflects the implementation status and recorded metrics available during preparation. Items identified as incomplete should only be updated after the corresponding changes have been implemented and verified.